

**EXERCISE I**

In the context of deep neural networks

**I.a) explain the concept of Loss function.**

The loss function is the function that computes the distance between the current output of the algorithm and the expected output. It's a method to evaluate how your algorithm models the data. It can be categorized into two groups. One for classification (discrete values, 0,1,2...) and the other for regression (continuous values).

**I.b) Consider the multiclass SVM loss (right).**

Compute the loss value for a dataset with four samples and a classification problem with four classes C0, C1, C2, C3. Sample S0 has label C0, and its label scores are (1.1, 2, 0.1, 0.5), S1 has label C3 and its class scores are (2, 1.5, 0.1, 3), S2 has label C1 and its class scores are (2, 3, 1, 1), S3 has label C2 and its class scores are (0.2, 3, 1, 2)

**Multiclass SVM loss:**

Given an example  $(x_i, y_i)$   
where  $x_i$  is the image and  
where  $y_i$  is the (integer) label,

and using the shorthand for the scores vector:  $s = f(x_i, W)$

the SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & \text{if } s_{y_i} \geq s_j + 1 \\ s_j - s_{y_i} + 1 & \text{otherwise} \end{cases}$$

$$= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

|         | Sample 0 | Sample 1 | Sample 2 | Sample 3 |
|---------|----------|----------|----------|----------|
| Class 0 | 1.1      | 2        | 2        | 0.2      |
| Class 1 | 2        | 1.5      | 3        | 3        |
| Class 2 | 0.1      | 0.1      | 1        | 1        |
| Class 3 | 0.5      | 3        | 1        | 2        |

- For sample S0 with label C0 and scores (1.1, 2, 0.1, 0.5):

-  $L_0 = \max(0, 2 - 1.1 + 1) + \max(0, 0.1 - 1.1 + 1) + \max(0, 0.5 - 1.1 + 1)$

-  $L_0 = \max(0, 1.9) + \max(0, 0) + \max(0, 0.4)$

-  $L_0 = 1.9 + 0 + 0.4 = 2.3$

- For sample S1 with label C3 and scores (2, 1.5, 0.1, 3):

-  $L_1 = \max(0, 2 - 3 + 1) + \max(0, 1.5 - 3 + 1) + \max(0, 0.1 - 3 + 1)$

-  $L_1 = \max(0, 0) + \max(0, 0) + \max(0, 0)$

-  $L_1 = 0 + 0 + 0 = 0$

- For sample S2 with label C1 and scores (2, 3, 1, 1):

-  $L_2 = \max(0, 2 - 3 + 1) + \max(0, 1 - 3 + 1) + \max(0, 1 - 3 + 1)$

-  $L_2 = \max(0, 0) + \max(0, 0) + \max(0, 0)$

-  $L_2 = 0 + 0 + 0 = 0$

- For sample S3 with label C2 and scores (0.2, 3, 1, 2):
- $L_3 = \max(0, 0.2 - 1 + 1) + \max(0, 3 - 1 + 1) + \max(0, 2 - 1 + 1)$
- $L_3 = \max(0, 0.2) + \max(0, 3) + \max(0, 2)$
- $L_3 = 0.2 + 3 + 2 = 5.2$

So, with (  $\Delta = 1$  ), the updated loss values for the given samples using ASCII characters are:

- $L_0 = 2.3$
- $L_1 = 0$
- $L_2 = 0$
- $L_3 = 5.2$

**$L_{tot} = 7.5$**

**I.c) give an example of class scores for C0, ...C3 for which the SVM loss has value 0.**

if the given example is classified correctly, then the scores of the incorrect classes are lower than the scores of the correct classes. this leads to a loss for the particular sample =0.

an example

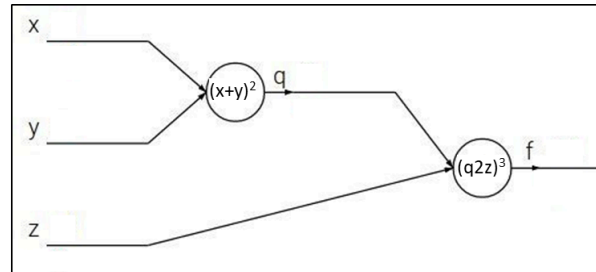
- S0 with label C0: (3, 2, 1, 0)
- S1 with label C3: (0, 1, 2, 3)
- S2 with label C1: (0, 3, 1, 2)
- S3 with label C2: (0, 1, 3, 2)

With correct samples highlighted in blue

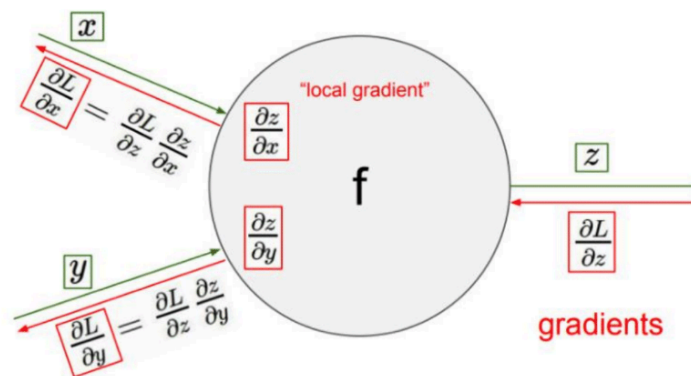
## EXERCISE II

Consider the following data flow graph representing the function to the left:

$$f(x, y, z) = ((x + y)^2 2z)^3$$



II.a) Compute the partial derivatives on all edges of the nodes using backpropagation



Source: Stanford <https://www.youtube.com/watch?v=i94OvYb6noo> (also in lecture slides)

$$q = (x+y)**2$$

$$f = (q2z)**3$$

$$dq/dx = 2x + 2y$$

$$dq/dy = 2y + 2x$$

$$df/dq = 3*(q)**2 * (2z)**3 = 24 * q**2 * z**3$$

$$df/dx = df/dq * dq/dx = (3*q**2 * (2z)**3) * (2x + 2y) = 48 * q**2 * z**3 * (x+y)$$

$$dq/dy = df/dq * dq/dy = (3*q**2 * (2z)**3) * (2x + 2y) = 48 * q**2 * z**3 * (x+y)$$

$$df/dz = 3 * z**2 * (2q)**3 = 24 * z**2 * q**3$$

**II.b) compute the value of the gradients for  $(x,y,z)=(1, 2, 3)$**

$$q = 9$$

$$f = (9 \cdot 2 \cdot 3)^3 = 157464$$

$$dq/dx = 2x + 2y = 2 + 4 = 6$$

$$dq/dy = 2y + 2x = 2 + 4 = 6$$

$$df/dq = 3 \cdot (q)^2 \cdot (2z)^3 = 3 \cdot 81 \cdot 6^3 \cdot 6 = 157464$$

$$df/dz = 24 \cdot z^2 \cdot q^3 = 157464$$

$$df/dx = df/dq \cdot dq/dx = 157464 \cdot 6 = 314928$$

$$df/dy = df/dq \cdot dq/dy = 157464 \cdot 6 = 314928$$

therefore, the gradients are

$$df/dx = 314928$$

$$df/dy = 314928$$

$$df/dz = 157464$$

**II.c) compute the gradient in the same point as II.b when  $q=x-2y$  and  $f=z-2q$**

$$(x,y,z)=(1, 2, 3)$$

$$q = 1 - 4 = -3$$

$$f = 3 - 2 \cdot (-3) = 3 + 6 = 9$$

$$dq/dx = 1$$

$$dq/dy = -2$$

$$df/dq = -2$$

$$df/dz = 1$$

$$df/dx = df/dq \cdot dq/dx = -2 \cdot 1 = -2 \rightarrow df/dx = -2$$

$$df/dy = df/dq \cdot dq/dy = -2 \cdot (-2) = 4 \rightarrow df/dy = 4$$

therefore the gradients are

$$df/dx = -2$$

$$df/dy = 4$$

$$df/dz = 1$$

### EXERCISE III

A parallel cluster of RISC-V processors is executing a convolutional layer with 8 input channels and 19 output channels. Input spatial size is 61x61. Inputs, outputs and weights are quantized to 8 bits. Convolutional filter size is 5x5, padding with zeros is used, stride=2, no bias. RELU non-linearity is applied in place on the output. After RELU, results are average-pooled (2x2 pooling window, stride=2). Assume that on average a RISC-V processor can perform a MAC in 4 cycles and a RELU operation in 6 cycles and a division in 12 cycles). The three kernels (convolution, relu, average pooling) are parallelized over the output channels. No parallelization overhead

**III.a) Compute the L1 minimal shared memory size that is needed to store the complete layer (input and output channels as well as weights) in L1**

Conv

| Convolution |    |         |       |        |                         |
|-------------|----|---------|-------|--------|-------------------------|
| Input       |    | Kernel  |       | Output |                         |
| W_in        | 61 | size    | K (W) | 5      | W_out [(W_in-K+2P)/S]+1 |
| H_in        | 61 |         | K (H) | 5      | H_out [(H_in-K+2P)/S]+1 |
| C_in        | 8  |         | C_in  | 8      | C_out                   |
|             |    |         | C_out | 19     | 19                      |
|             |    | stride  | S_w   | 2      |                         |
|             |    |         | S_h   | 2      |                         |
|             |    | padding | P     | 2      |                         |

| Memory occupation |                  |                     |          |
|-------------------|------------------|---------------------|----------|
| Input             | Kernel           | Output              | TOT size |
| W_in*H_in*C_in =  | K*K*C_in*C_out = | W_out*H_out*C_out = | In+K+Out |
| 29768 B           | 3800 B           | 18259 B             | 51827 B  |

Avg pool

| AVG POOL |    |          |   |        |    |
|----------|----|----------|---|--------|----|
| Input    |    | AVG Pool |   | Output |    |
| W_out    | 31 | K (W)    | 2 | W_out  | 15 |
| H_out    | 31 | K (H)    | 2 | H_out  | 15 |
| C_out    | 19 | stride   | 2 | C_out  | 19 |
|          |    | pad      | 0 |        |    |

| Memory occupation |        |                     |          |
|-------------------|--------|---------------------|----------|
| 5x5 conv          | Kernel | Output              | TOT size |
|                   |        | W_out*H_out*C_out = | In+K+Out |
| 51827 B           | 0 B    | 4275 B              | 56102 B  |

tot = input\_conv + weights\_conv + output\_conv + output\_maxpool = **56102B**

### III.b) Compute the total number of cycles to process the layer on a 4-core cluster

|      | cycles    | Processed by |       |
|------|-----------|--------------|-------|
| MAC  | 4 cycles  | 4            | Cores |
| ReLU | 6 cycles  | 4            | Core  |
| Div  | 12 cycles | 4            | Core  |

Cycles\_conv (4 cores) = (Hout \* Wout \* Cin \* ceil(Cout/Ncores) \* K \* K) \* [MAC] cycles = 3844000 cycles  
 Cycles\_ReLU (4 core) = ( ceil(Cout/N\_cores) \*Wout\*Hout ) \* 3 cycles =  
 = (19/N\_cores\*126\*126)\* [ReLU] cycles = 28830 cycles  
 Cycles\_Avgpool (4 core) = SUM \* 4 cycles + DIV \* 12 cycles =  
 = (3)\*Wout\*Hout\*ceil(Cout/Ncores) \*4cycles + (1)\*Wout\*Hout\*ceil(Cout/Ncores) \* 12 cycles =  
 13500 + 13500 27000 cycles  
 Cycles\_tot = 3899830 cycles

### III.c) Compute the speedup obtained when processing the layer with 4 cores versus only 1 core (sequential).

|      | cycles    | Processed by |       |
|------|-----------|--------------|-------|
| MAC  | 4 cycles  | 1            | Cores |
| ReLU | 6 cycles  | 1            | Core  |
| Div  | 12 cycles | 1            | Core  |

Cycles\_conv (4 cores) = (Hout \* Wout \* Cin \* ceil(Cout/Ncores) \* K \* K) \* [MAC] cycles = 14607200 cycles  
 Cycles\_ReLU (4 core) = ( ceil(Cout/N\_cores) \*Wout\*Hout ) \* 3 cycles =  
 = (19/N\_cores\*126\*126)\* [ReLU] cycles = 109554 cycles  
 Cycles\_Avgpool (4 core) = SUM \* 4 cycles + DIV \* 12 cycles =  
 = (3)\*Wout\*Hout\*ceil(Cout/Ncores) \*4cycles + (1)\*Wout\*Hout\*ceil(Cout/Ncores) \* 12 cycles =  
 51300 + 51300 102600 cycles  
 Cycles\_tot = 14819354 cycles

speedup = 3899830 / 14819354 = **3.8 x**